CNN_981.ipynb

File Edit View Insert Runtime Tools Help All changes saved

Comment Share

+ Code + Text

T4 RAM Disk Colab AI

[2] ! pip install kaggle
from google.colab import drive
drive.mount('/content/drive')
! mkdir ~/.kaggle
!cp kaggle.json ~/.kaggle
!chmod 600 ~/.kaggle/kaggle.json
! kaggle datasets download 'paultimothymooney/breast-histopathology-images'
! unzip breast-histopathology-images.zip

```
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1751_y951_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1801_y1001_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1801_y1051_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1801_y601_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1801_y651_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1801_y701_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1801_y751_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1801_y801_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1801_y851_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1801_y901_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1801_y951_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1851_y1001_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1851_y1051_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1851_y601_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1851_y651_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1851_y701_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1851_y751_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1851_y801_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1851_y851_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1851_y901_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1851_y951_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1901_y1001_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1901_y1051_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1901_y601_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1901_y651_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1901_y701_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1901_y751_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1901_y801_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1901_y851_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1901_y901_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1901_y951_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1951_y1001_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1951_y1051_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1951_y601_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1951_y651_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1951_y701_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1951_y751_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1951_y801_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1951_y851_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1951_y901_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1951_y951_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x2001_y1001_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x2001_y1051_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x2001_y701_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x2001_y751_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x2001_y801_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x2001_y851_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x2001_y901_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x2001_y951_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x2051_y1001_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x2051_y1051_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x2051_y801_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x2051_y851_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x2051_y901_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x2051_y951_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x2101_y1001_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x2101_y901_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x2101_y951_class1.png
```

[3] import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import cv2
import glob
import random
from sklearn.metrics import classification_report
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout, BatchNormalization
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.callbacks import EarlyStopping
from tensorflow.keras.utils import to_categorical
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D

[4] from google.colab import drive
drive.mount('/content/drive')

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
```

[5] breast_img = glob.glob('/content/IDC_regular_ps50_idx5/**/*.png', recursive = True)

for imgname in breast_img[:3]:
    print(imgname)

```
/content/IDC_regular_ps50_idx5/9174/1/9174_idx5_x1551_y901_class1.png
/content/IDC_regular_ps50_idx5/9174/1/9174_idx5_x1401_y1101_class1.png
/content/IDC_regular_ps50_idx5/9174/1/9174_idx5_x1501_y801_class1.png
```

```
[6] # Split images based on class (IDC negative and IDC positive)
    N_IDC = [img for img in breast_img if img[-5] == '0']
    P_IDC = [img for img in breast_img if img[-5] == '1']
```

```
[7] # Reduce the number of IDC (-) samples to balance the classes
    NewN_IDC = N_IDC[:78786]
```

```
[8] # Data Preprocessing
    non_img_arr = []
    can_img_arr = []
```

```
[9] # Process IDC (-) images
    for img in NewN_IDC:
        n_img = cv2.imread(img, cv2.IMREAD_COLOR)
        n_img_size = cv2.resize(n_img, (50, 50), interpolation=cv2.INTER_LINEAR)
        non_img_arr.append([n_img_size, 0])
```

```
[10] # Process IDC (+) images
    for img in P_IDC:
        c_img = cv2.imread(img, cv2.IMREAD_COLOR)
        c_img_size = cv2.resize(c_img, (50, 50), interpolation=cv2.INTER_LINEAR)
        can_img_arr.append([c_img_size, 1])
```

```
[11] # Combine the processed images and shuffle
    breast_img_arr = np.concatenate((non_img_arr[:12389], can_img_arr[:12389]))
    random.shuffle(breast_img_arr)

    X = np.array([feature for feature, _ in breast_img_arr])
    y = np.array([label for _, label in breast_img_arr])
```

```
<__array_function__ internals>:180: VisibleDeprecationWarning: Creating an ndarray from ragged nested sequences (which is a list-or-tuple of lists-or-tuples-or ndarrays with different l
```

```
[12] # Train-test split
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3)
```

```
[13] # Normalize pixel values
    X_train = X_train / 255.0
    X_test = X_test / 255.0
```

```
[14] # Model Building
    model = Sequential([
        Conv2D(32, (3, 3), activation='relu', kernel_initializer='he_uniform', padding='same', input_shape=(50, 50, 3)),
        BatchNormalization(),
        MaxPooling2D((2, 2)),
        Dropout(0.3),

        Conv2D(64, (3, 3), activation='relu', kernel_initializer='he_uniform', padding='same'),
        BatchNormalization(),
        MaxPooling2D((2, 2)),
        Dropout(0.3),

        Conv2D(128, (3, 3), activation='relu', kernel_initializer='he_uniform', padding='same'),
        BatchNormalization(),
        MaxPooling2D((2, 2)),
        Dropout(0.3),

        Flatten(),
        Dense(128, activation='relu', kernel_initializer='he_uniform'),
        BatchNormalization(),
        Dropout(0.5),
        Dense(1, activation='sigmoid')
    ])
```

```
[15] from tensorflow.keras.preprocessing.image import ImageDataGenerator

    # Augment data during training
    train_datagen = ImageDataGenerator(
        rotation_range=20,
        width_shift_range=0.2,
        height_shift_range=0.2,
        shear_range=0.2,
        zoom_range=0.2,
        horizontal_flip=True,
        vertical_flip=True,
        fill_mode='nearest'
    )

    train_generator = train_datagen.flow(X_train, y_train, batch_size=35)
```

```
[16] # Compile the model
    model.compile(optimizer=Adam(learning_rate=0.0001), loss='binary_crossentropy', metrics=['accuracy'])
    model.summary()
```

```
Model: "sequential"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 conv2d (Conv2D)             (None, 50, 50, 32)        896

 batch_normalization (Batch  (None, 50, 50, 32)        128
 Normalization)

 max_pooling2d (MaxPooling2  (None, 25, 25, 32)        0
 D)

 dropout (Dropout)           (None, 25, 25, 32)        0

 conv2d_1 (Conv2D)           (None, 25, 25, 64)        18496
```

```
batch_normalization_1 (Bat  (None, 25, 25, 64)        256
chNormalization)

max_pooling2d_1 (MaxPoolin  (None, 12, 12, 64)        0
g2D)

dropout_1 (Dropout)         (None, 12, 12, 64)        0

conv2d_2 (Conv2D)           (None, 12, 12, 128)       73856

batch_normalization_2 (Bat  (None, 12, 12, 128)       512
chNormalization)

max_pooling2d_2 (MaxPoolin  (None, 6, 6, 128)         0
g2D)

dropout_2 (Dropout)         (None, 6, 6, 128)         0

flatten (Flatten)           (None, 4608)              0

dense (Dense)               (None, 128)               589952

batch_normalization_3 (Bat  (None, 128)               512
chNormalization)

dropout_3 (Dropout)         (None, 128)               0

dense_1 (Dense)             (None, 1)                 129

=================================================================
Total params: 684737 (2.61 MB)
Trainable params: 684033 (2.61 MB)
Non-trainable params: 704 (2.75 KB)
_________________________________________________________________
```

19m

```
[17] # Model Training
history = model.fit(X_train, y_train, validation_data=(X_test, y_test), epochs=200, batch_size=35 )
```

```
Epoch 172/200
496/496 [==============================] - 6s 12ms/step - loss: 0.0348 - accuracy: 0.9879 - val_loss: 0.1699 - val_accuracy: 0.9661
Epoch 173/200
496/496 [==============================] - 5s 10ms/step - loss: 0.0406 - accuracy: 0.9850 - val_loss: 0.1771 - val_accuracy: 0.9638
Epoch 174/200
496/496 [==============================] - 6s 12ms/step - loss: 0.0341 - accuracy: 0.9875 - val_loss: 0.1959 - val_accuracy: 0.9580
Epoch 175/200
496/496 [==============================] - 6s 12ms/step - loss: 0.0355 - accuracy: 0.9880 - val_loss: 0.1997 - val_accuracy: 0.9591
Epoch 176/200
496/496 [==============================] - 5s 11ms/step - loss: 0.0376 - accuracy: 0.9870 - val_loss: 0.1865 - val_accuracy: 0.9614
Epoch 177/200
496/496 [==============================] - 6s 12ms/step - loss: 0.0378 - accuracy: 0.9853 - val_loss: 0.1840 - val_accuracy: 0.9600
Epoch 178/200
496/496 [==============================] - 6s 12ms/step - loss: 0.0380 - accuracy: 0.9860 - val_loss: 0.1866 - val_accuracy: 0.9635
Epoch 179/200
496/496 [==============================] - 5s 11ms/step - loss: 0.0344 - accuracy: 0.9873 - val_loss: 0.1911 - val_accuracy: 0.9609
Epoch 180/200
496/496 [==============================] - 6s 12ms/step - loss: 0.0389 - accuracy: 0.9864 - val_loss: 0.1578 - val_accuracy: 0.9660
Epoch 181/200
496/496 [==============================] - 6s 12ms/step - loss: 0.0320 - accuracy: 0.9890 - val_loss: 0.1859 - val_accuracy: 0.9637
Epoch 182/200
496/496 [==============================] - 5s 10ms/step - loss: 0.0358 - accuracy: 0.9866 - val_loss: 0.1530 - val_accuracy: 0.9664
Epoch 183/200
496/496 [==============================] - 6s 12ms/step - loss: 0.0340 - accuracy: 0.9884 - val_loss: 0.1989 - val_accuracy: 0.9652
Epoch 184/200
496/496 [==============================] - 6s 12ms/step - loss: 0.0351 - accuracy: 0.9875 - val_loss: 0.1745 - val_accuracy: 0.9638
Epoch 185/200
496/496 [==============================] - 5s 10ms/step - loss: 0.0332 - accuracy: 0.9880 - val_loss: 0.1833 - val_accuracy: 0.9635
Epoch 186/200
496/496 [==============================] - 7s 13ms/step - loss: 0.0352 - accuracy: 0.9871 - val_loss: 0.1612 - val_accuracy: 0.9660
Epoch 187/200
496/496 [==============================] - 7s 15ms/step - loss: 0.0335 - accuracy: 0.9866 - val_loss: 0.2082 - val_accuracy: 0.9609
Epoch 188/200
496/496 [==============================] - 6s 12ms/step - loss: 0.0333 - accuracy: 0.9884 - val_loss: 0.1548 - val_accuracy: 0.9658
Epoch 189/200
496/496 [==============================] - 7s 14ms/step - loss: 0.0324 - accuracy: 0.9882 - val_loss: 0.1522 - val_accuracy: 0.9673
Epoch 190/200
496/496 [==============================] - 5s 11ms/step - loss: 0.0337 - accuracy: 0.9876 - val_loss: 0.2071 - val_accuracy: 0.9625
Epoch 191/200
496/496 [==============================] - 5s 10ms/step - loss: 0.0329 - accuracy: 0.9884 - val_loss: 0.1830 - val_accuracy: 0.9619
Epoch 192/200
496/496 [==============================] - 6s 13ms/step - loss: 0.0304 - accuracy: 0.9885 - val_loss: 0.1546 - val_accuracy: 0.9677
Epoch 193/200
496/496 [==============================] - 5s 11ms/step - loss: 0.0273 - accuracy: 0.9908 - val_loss: 0.1718 - val_accuracy: 0.9658
Epoch 194/200
496/496 [==============================] - 5s 11ms/step - loss: 0.0322 - accuracy: 0.9885 - val_loss: 0.1635 - val_accuracy: 0.9670
Epoch 195/200
496/496 [==============================] - 6s 13ms/step - loss: 0.0325 - accuracy: 0.9888 - val_loss: 0.1586 - val_accuracy: 0.9650
Epoch 196/200
496/496 [==============================] - 5s 11ms/step - loss: 0.0298 - accuracy: 0.9893 - val_loss: 0.1589 - val_accuracy: 0.9685
Epoch 197/200
496/496 [==============================] - 5s 11ms/step - loss: 0.0328 - accuracy: 0.9880 - val_loss: 0.2002 - val_accuracy: 0.9658
Epoch 198/200
496/496 [==============================] - 6s 13ms/step - loss: 0.0326 - accuracy: 0.9885 - val_loss: 0.1665 - val_accuracy: 0.9685
Epoch 199/200
496/496 [==============================] - 5s 11ms/step - loss: 0.0311 - accuracy: 0.9890 - val_loss: 0.1883 - val_accuracy: 0.9674
Epoch 200/200
496/496 [==============================] - 5s 10ms/step - loss: 0.0282 - accuracy: 0.9905 - val_loss: 0.1594 - val_accuracy: 0.9674
```

3s

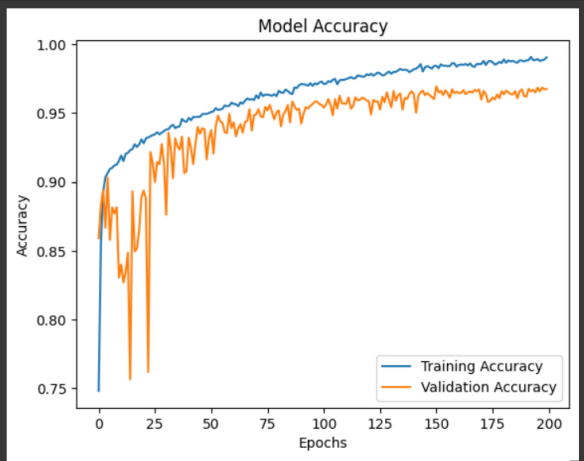
```
[18] # Model Evaluation
model.evaluate(X_test, y_test)
```

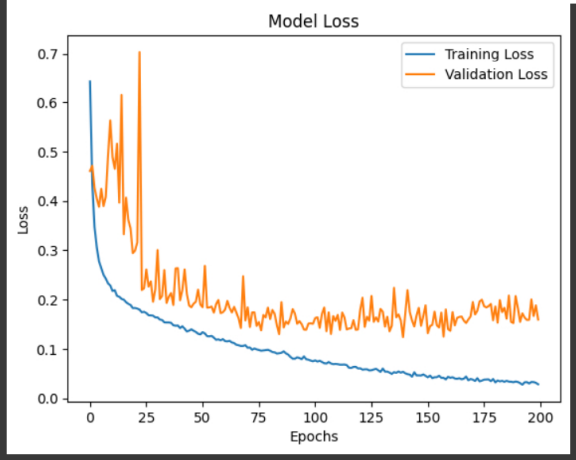
```
233/233 [==============================] - 2s 5ms/step - loss: 0.1594 - accuracy: 0.9674
[0.15939421951770782, 0.9674468636512756]
```

1s

```
[19] # Plotting Accuracy and Loss
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title('Model Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
plt.show()
```

```
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('Model Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.show()
```





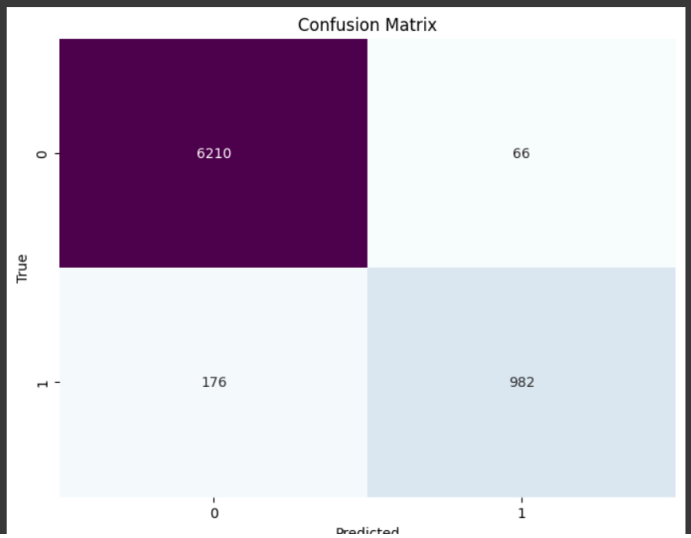
[20]
```
# Predict probabilities for the test set
y_pred_prob = model.predict(X_test)

# Convert probabilities to classes
y_pred = (y_pred_prob > 0.5).astype('int32')

# Create confusion matrix
cm = confusion_matrix(y_test, y_pred)
```

```
233/233 [==============================] - 1s 3ms/step
```

[21]
```
# Plot Confusion Matrix
plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='BuPu', cbar=False)
plt.xlabel('Predicted')
plt.ylabel('True')
plt.title('Confusion Matrix')
plt.show()
```

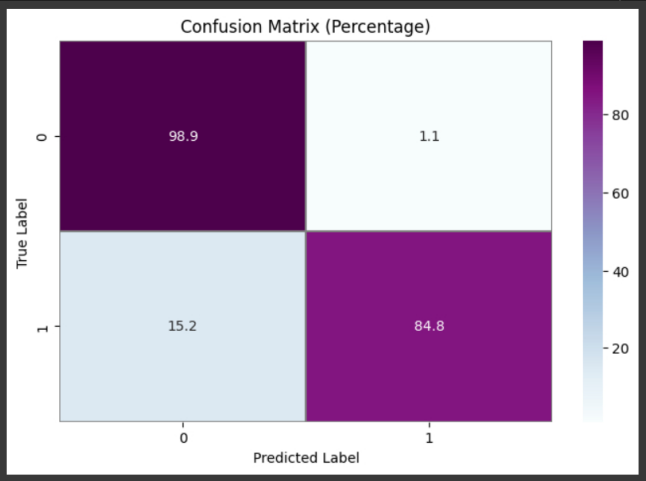


```
[22] from sklearn.metrics import accuracy_score, confusion_matrix


     confusion_mtx = confusion_matrix(y_test, y_pred)

     # calculate the percentage
     confusion_mtx_percent = confusion_mtx.astype('float') / confusion_mtx.sum(axis=1)[:, np.newaxis] * 100

     f, ax = plt.subplots(figsize=(8, 5))
     sns.heatmap(confusion_mtx_percent, annot=True, linewidths=0.01, cmap="BuPu", linecolor="gray", fmt='.1f', ax=ax)
     plt.xlabel("Predicted Label")
     plt.ylabel("True Label")
     plt.title("Confusion Matrix (Percentage)")
     plt.show()
```




```
[23] # Save the Model
     model.save("Brest_idc_Model2.h5")
```

```
/usr/local/lib/python3.10/dist-packages/keras/src/engine/training.py:3103: UserWarning: You are saving your model as an HDF5 file via `model.save()`. This file format is considered lega
  saving_api.save_model(
```

```
[24] import cv2
     import numpy as np
     from tensorflow.keras.models import load_model
     from google.colab import files

     # Load the saved model
     model = load_model('/content/Brest_idc_Model2.h5')

     # Function to preprocess the uploaded image
     def preprocess_image(image_path):
         img = cv2.imread(image_path)
         img = cv2.resize(img, (50, 50))  # Resize image to match model input shape
         img = np.expand_dims(img, axis=0)  # Add batch dimension
         img = img / 255.0  # Normalize pixel values
         return img

     # Function to make predictions
     def predict_image(image_path):
         img = preprocess_image(image_path)
         prediction = model.predict(img)
         if prediction[0][0] > 0.5:  # Assuming binary classification
             return "IDC (+)"
         else:
             return "IDC (-)"

     # Upload multiple images for testing
     uploaded = files.upload()

     # Perform prediction on each uploaded image
     for image_name in uploaded.keys():
         image_path = image_name
         prediction = predict_image(image_path)
         print(f"The predicted class for {image_name} is: {prediction}")
```

```
Choose Files  5 files
```

- **8864_idx5_x1451_y2601_class1.png**(image/png) - 6394 bytes, last modified: 9/25/2019 - 100% done
- **8864_idx5_x1451_y2651_class1.png**(image/png) - 6440 bytes, last modified: 9/25/2019 - 100% done
- **8864_idx5_x1451_y2701_class1.png**(image/png) - 6488 bytes, last modified: 9/25/2019 - 100% done
- **8864_idx5_x1451_y2751_class1.png**(image/png) - 6533 bytes, last modified: 9/25/2019 - 100% done
- **8864_idx5_x1451_y2801_class1.png**(image/png) - 3565 bytes, last modified: 9/25/2019 - 100% done

```
Saving 8864_idx5_x1451_y2601_class1.png to 8864_idx5_x1451_y2601_class1.png
Saving 8864_idx5_x1451_y2651_class1.png to 8864_idx5_x1451_y2651_class1.png
Saving 8864_idx5_x1451_y2701_class1.png to 8864_idx5_x1451_y2701_class1.png
Saving 8864_idx5_x1451_y2751_class1.png to 8864_idx5_x1451_y2751_class1.png
Saving 8864_idx5_x1451_y2801_class1.png to 8864_idx5_x1451_y2801_class1.png
1/1 [==============================] - 0s 390ms/step
The predicted class for 8864_idx5_x1451_y2601_class1.png is: IDC (+)
1/1 [==============================] - 0s 28ms/step
The predicted class for 8864_idx5_x1451_y2651_class1.png is: IDC (+)
1/1 [==============================] - 0s 26ms/step
The predicted class for 8864_idx5_x1451_y2701_class1.png is: IDC (+)
1/1 [==============================] - 0s 27ms/step
The predicted class for 8864_idx5_x1451_y2751_class1.png is: IDC (+)
1/1 [==============================] - 0s 28ms/step
The predicted class for 8864_idx5_x1451_y2801_class1.png is: IDC (-)
```

```
[25] from sklearn.metrics import accuracy_score

     # Assuming y_pred is your predicted labels for the test set
     accuracy = accuracy_score(y_test, y_pred)
     print(f"Test Accuracy: {accuracy * 100:.2f}%")
```

```
Test Accuracy: 96.74%
```

```
# Get predicted probabilities for the positive class
y_pred_prob = model.predict(X_test)

# For binary classification, consider the probability of the positive class (class 1)
y_pred_prob_positive = y_pred_prob.ravel()

# Calculate ROC curve
fpr, tpr, thresholds = roc_curve(y_test, y_pred_prob_positive)

# Calculate ROC AUC score
roc_auc = roc_auc_score(y_test, y_pred_prob_positive)

# Plot ROC curve
plt.figure(figsize=(8, 6))
plt.plot(fpr, tpr, color='blue', lw=2, label=f'ROC Curve (AUC = {roc_auc:.2f})')
plt.plot([0, 1], [0, 1], color='red', linestyle='--')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic (ROC) Curve')
plt.legend()
plt.show()
```

```
233/233 [==============================] - 1s 3ms/step
```